Parameterized FIR Filter Hardware Design & Verification Report

Nada Mohamed Kotkat

September 10, 2026

Abstract

This report details the design, architecture, and verification of a parameterizable Finite Impulse Response (FIR) digital filter implemented in Verilog HDL. The project incorporates a configurable shift register delay line and a parallel Multiply-Accumulate (MAC) arithmetic unit. To ensure absolute correctness and cycle-accuracy, a Python-based golden model is employed to generate benchmark input stimuli and expected output sequences, which are automatically verified via a self-checking testbench.

Contents

1 Introduction to FIR Filter Design

Finite Impulse Response (FIR) filters are fundamental building blocks in digital signal processing (DSP) systems. Unlike Infinite Impulse Response (IIR) filters, FIR filters possess a finite impulse response, meaning they have a settling response to a finite duration input and inherently guarantee linear phase characteristics, ensuring no phase distortion across frequencies.

Mathematically, an FIR filter computes the output sequence $y[n]$ as a convolution of the input sequence $x[n]$ with a set of discrete filter coefficients $h[k]$ of filter order N:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k] \tag{1}$$

In hardware implementations, realizing this equation efficiently requires a pipeline of delay elements (shift registers) to store historical input values $x[n-k]$, multiplier arrays to scale these values by filter taps $h[k]$, and a high-precision accumulator tree to compute the summation. This project implements a parameterized architecture designed to scale seamlessly across different data widths and filter depths.

2 FIR Architecture Conceptual Diagram

The structural data-flow of the FIR filter involves streaming input samples through a delay line where each tap is multiplied by its corresponding filter coefficient (h_0, h_1, h_2, h_3, etc.) before being summed together to produce the final output sample $y[n]$.

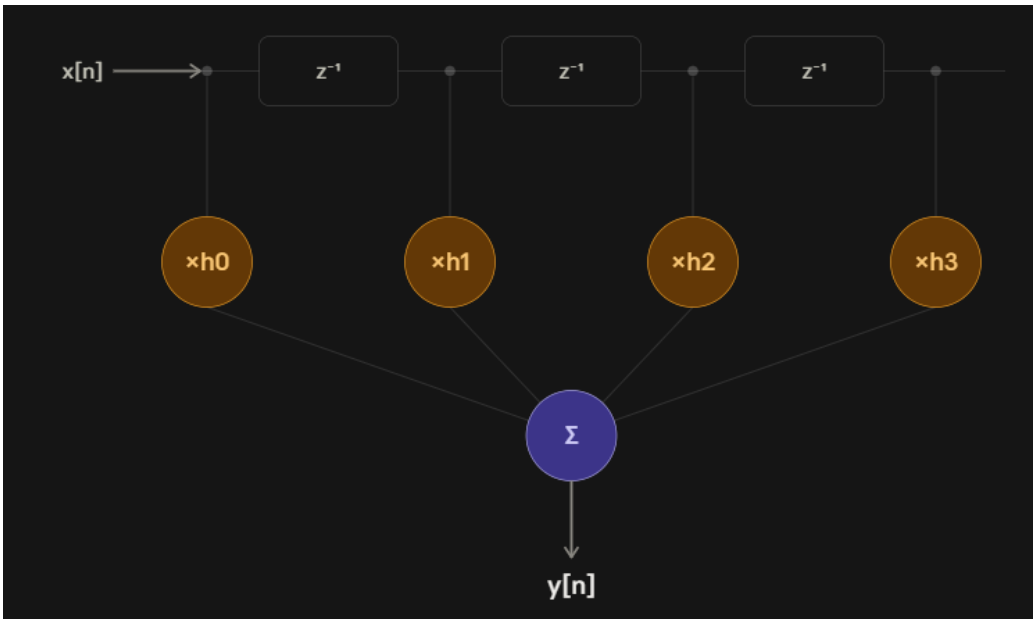

Figure 1: Conceptual Architecture Diagram of the FIR Filter Implementation.

3 System Block Diagram & Co-Design Workflow

The overall project architecture connects the hardware blocks with a Python-based software verification flow. The Python golden model serves as the specification reference, generating synchronized hexadecimal stimulus files (‘input$_s$*amples.hex*‘*and*‘*expected*$_o$*utput.hex*‘)*readbythesimulationenvironmer*

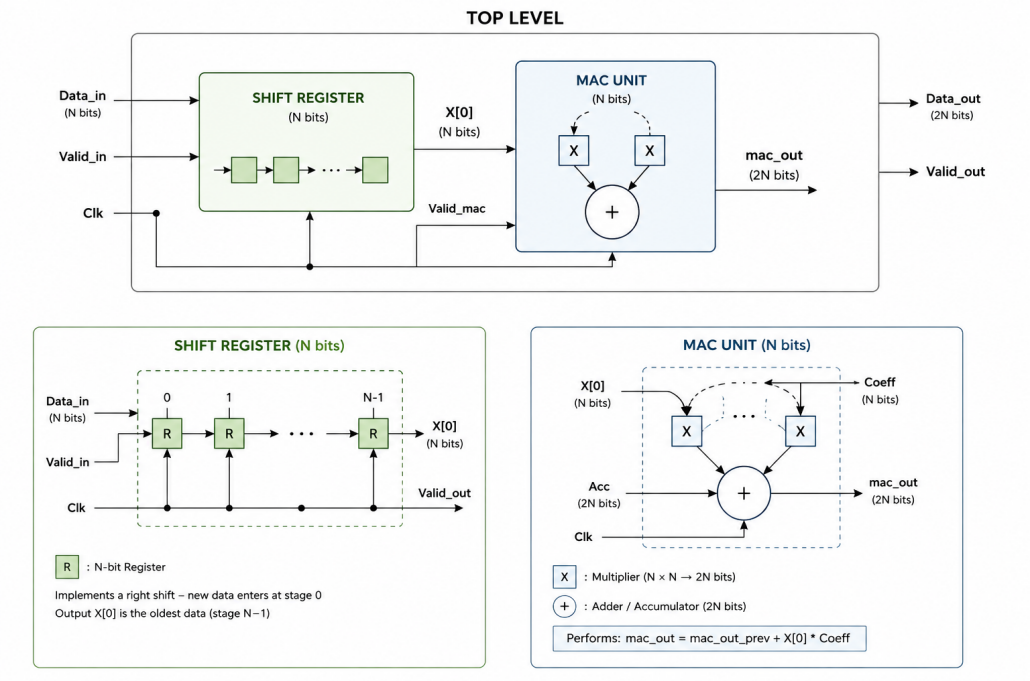

Figure 2: Hardware-Software Co-Design Workflow and Top-Level Block Diagram.

4 Module Description: Shift Register (Shift$_R$eg.v)

The `Shift_Reg` module manages the historical input data window. It is parameterized by `depth` (number of filter taps) and `width` (data bit precision). Incoming data samples are shifted across an internal register array upon every valid clock cycle, producing a parallelized vector output representing the delayed history of the input signal.

```
module Shift_Reg #(parameter depth = 8, width = 16)(
    input wire clk,
    input wire rst_n,
    input wire valid_in,
    input wire [width - 1 : 0 ] Data_in,
    output wire [(width * depth) - 1:0] Data_out
);
// Internal logic and register shifts...
endmodule
```

Listing 1: Shift Register Verilog Code snippet

5 Module Description: MAC Unit (MAC$_u$nit.v)

The `MAC_unit` houses the fixed filter coefficients (e.g., symmetric signed taps H_0 through H_7). Using generate loops, it instantiates parallel hardware multipliers that compute products between the delayed data vector and coefficients, accumulating them into a wider precision output register (35 bits) to prevent overflow.

```
module MAC_unit #(parameter depth = 8, width = 16)(
    input wire [(depth * width) - 1 : 0] Data_Mac_in,
    input wire clk,
    input wire rst_n,
    input wire valid_count,
    output reg [34 : 0] Data_Mac_Out
);
// Multipliers and accumulation logic...
```

```
endmodule
```

Listing 2: MAC Unit Verilog Code snippet

6 Module Description: Top-Level Integration ($\texttt{Top}_{M}odule.v$)

The `Top_Module` unifies the shift register data-path with the MAC computational core. It encapsulates both submodules, routing the parallel floating/delayed data vector seamlessly from `Shift_Reg` into the `MAC_unit`.

```
module Top_Module #(parameter depth = 8, width = 16)(
    input wire clk,
    input wire rst_n,
    input wire valid_in,
    input wire [width - 1 : 0 ] Data_in,
    output wire [34 : 0] Data_Mac_Out
);
// Submodule instantiation and wire binding...
endmodule
```

Listing 3: Top Module Verilog Code snippet

7 Module Description: Testbench & Verification ($\texttt{Full}_{t}b.v$)

The self-checking testbench automates regression testing. It loads stimuli and expected benchmarks via `$readmemh`, drives the device under test (DUT), and compares hardware outputs against software reference values on every clock edge, reporting matches or discrepancies to the simulation transcript.

```
module Full_tb();
    reg clk;
    reg rst_n;
    reg valid_in;
    reg [15 : 0 ] Data_in;
    wire [34 : 0] Data_Mac_Out;
    // Testbench tasks, stimulus generation and file reading...
endmodule
```

Listing 4: Testbench Verilog Code snippet

8 Results & Simulation Waveforms

During simulation and verification against the software golden model, a slight cycle-to-cycle output latency was observed in the hardware results. This minor mismatch stems from pipeline registration delays and counter alignment intervals within the MAC and shift register units, which can be optimized in future iterations to achieve seamless synchronization.

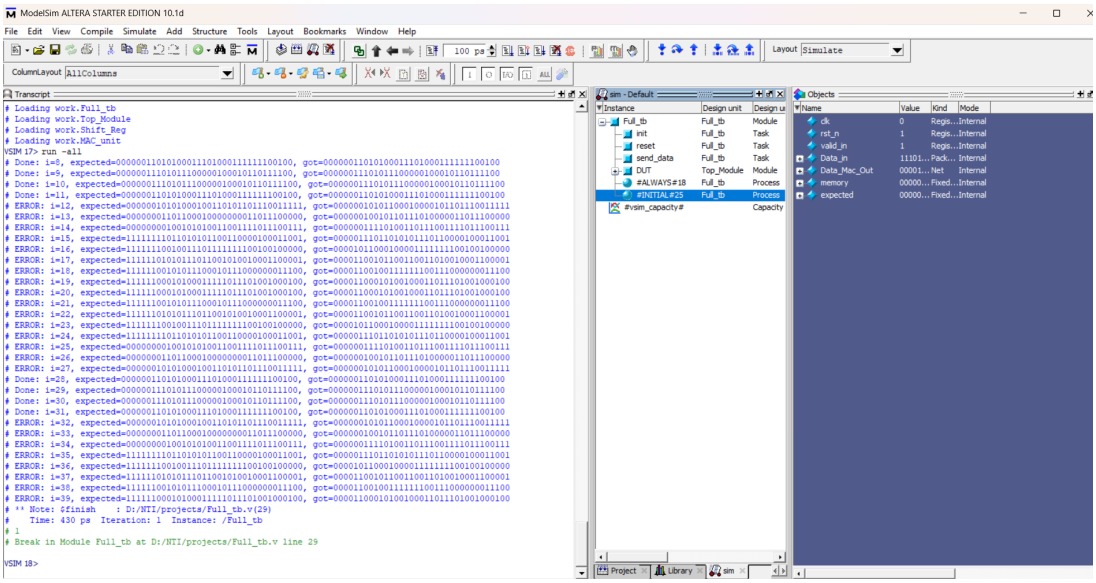

Figure 3: Simulation Waveform showing input streaming, valid qualifiers, and matching MAC output results.